

Pimpri Chinchwad College of Engineering, Pune
Department of Computer Engineering
AY 2025-26 Semester - VI
Course : Operating Systems [BCE26PE01]
Course Owner: Dr. Pravin S. Game

Note: These are questions for reference and practice. It is not necessary that the same/similar questions will come in SA exams. This is not an exhaustive list of questions.

Unit - 1 CO1 (Dr. Pravin S. Game) Compare various process scheduling algorithms for a given snapshot of the system		
Q. No.	Question	Level (L/M/H OTS)
1.	Differentiate between system calls and system programs with examples.	L
2.	Discuss how a time-sharing OS is different from a batch processing OS.	L
3.	Examine the effects of preemptive and non-preemptive scheduling on process states.	L
4.	How Operating Systems has evolved from batch systems to modern OS.	M
5.	How system calls facilitate communication between user applications and the OS.	M
6.	Compare multi-tasking, multi-programming and multi-processing Operating Systems.	M
7.	Describe time-sharing Operating Systems with a real-life example.	L
8.	Evaluate the efficiency of a Round Robin scheduling algorithm in handling process state transitions compared to Shortest Job Next (SJN).	L
9.	How does the Process Control Block (PCB) contribute to process switching in a multiprogramming system?	L
10.	Differentiate between user mode and kernel mode in an OS. Why is it necessary?	M
11.	How does an Operating System ensure security and protection? Discuss with examples.	M
12.	How does the OS perform error handling using system calls? Provide examples.	M

13.	Compare and contrast the process states "Ready" and "Blocked" in terms of CPU scheduling and resource allocation.	M																		
14.	Consider following processes with length of CPU burst time in milliseconds <table><tr><td>Process</td><td>Burst time</td></tr><tr><td>P1</td><td>5</td></tr><tr><td>P2</td><td>10</td></tr><tr><td>P3</td><td>2</td></tr><tr><td>P4</td><td>1</td></tr></table> All process arrived in order p1, p2, p3, p4 all time zero a) Draw Gantt charts illustrating execution of these processes for SJF and round robin (quantum=1) b) Calculate waiting time for each process for each scheduling algorithm c) Calculate average waiting time for each scheduling algorithm	Process	Burst time	P1	5	P2	10	P3	2	P4	1	M								
Process	Burst time																			
P1	5																			
P2	10																			
P3	2																			
P4	1																			
15.	Consider 3 processes P1, P2 and P3, which require 5, 7 and 4 time units and arrive at time 0, 1 and 3. Draw the Gantt chart, process completion sequence and average waiting time for. (i) Round robin scheduling with CPU quantum of 2 time units. (ii) FCFS.	M																		
16.	Consider following processes with length of CPU burst time in millisecond <table><tr><td>Process</td><td>Burst time</td><td>Priority</td></tr><tr><td>P1</td><td>10</td><td>3</td></tr><tr><td>P2</td><td>1</td><td>1</td></tr><tr><td>P3</td><td>2</td><td>3</td></tr><tr><td>P4</td><td>1</td><td>4</td></tr><tr><td>P5</td><td>5</td><td>2</td></tr></table> All processes arrived in order p1, p2, p3, p4, p5 all at time zero. 1) Draw Gantt charts illustrating execution of these processes for SJF-preemptive, non preemptive priority (smaller priority number implies a higher priority) & round robin(quantum=1) 2) Calculate turnaround time for each process for scheduling algorithm in part (1) 3) Calculate waiting time for each scheduling algorithm in part (1)	Process	Burst time	Priority	P1	10	3	P2	1	1	P3	2	3	P4	1	4	P5	5	2	H
Process	Burst time	Priority																		
P1	10	3																		
P2	1	1																		
P3	2	3																		
P4	1	4																		
P5	5	2																		
17.	“Priority inversion is a condition that occurs in real time systems where a low priority process is starved because higher priority processes have gained hold of the CPU” – Comment on this statement.	H																		
18.	Consider the following processes: <table><tr><td>Process</td><td>Arrival Time</td><td>Burst Time</td></tr><tr><td>P1</td><td>0 ms</td><td>7 ms</td></tr></table>	Process	Arrival Time	Burst Time	P1	0 ms	7 ms	M												
Process	Arrival Time	Burst Time																		
P1	0 ms	7 ms																		

	<table><tr><td>P2</td><td>1 ms</td><td>5 ms</td></tr><tr><td>P3</td><td>2 ms</td><td>1 ms</td></tr><tr><td>P4</td><td>3 ms</td><td>2 ms</td></tr></table> Compare Shortest Job First -Preemptive and FCFS scheduling algorithm based on average waiting time.	P2	1 ms	5 ms	P3	2 ms	1 ms	P4	3 ms	2 ms												
P2	1 ms	5 ms																				
P3	2 ms	1 ms																				
P4	3 ms	2 ms																				
19.	Consider the following processes with their priority values (lower number means higher priority): <table><tr><th>Process</th><th>Arrival Time</th><th>Burst Time</th><th>Priority</th></tr><tr><td>P1</td><td>0 ms</td><td>5 ms</td><td>2</td></tr><tr><td>P2</td><td>1 ms</td><td>3 ms</td><td>1</td></tr><tr><td>P3</td><td>2 ms</td><td>8 ms</td><td>4</td></tr><tr><td>P4</td><td>3 ms</td><td>6 ms</td><td>3</td></tr></table> Compare Non-Preemptive and preemptive versions of Priority Scheduling based on average turnaround time .	Process	Arrival Time	Burst Time	Priority	P1	0 ms	5 ms	2	P2	1 ms	3 ms	1	P3	2 ms	8 ms	4	P4	3 ms	6 ms	3	M
Process	Arrival Time	Burst Time	Priority																			
P1	0 ms	5 ms	2																			
P2	1 ms	3 ms	1																			
P3	2 ms	8 ms	4																			
P4	3 ms	6 ms	3																			
20.	Consider the following processes with a time quantum of 3 ms: <table><tr><th>Process</th><th>Arrival Time</th><th>Burst Time</th></tr><tr><td>P1</td><td>0 ms</td><td>10 ms</td></tr><tr><td>P2</td><td>1 ms</td><td>4 ms</td></tr><tr><td>P3</td><td>2 ms</td><td>5 ms</td></tr><tr><td>P4</td><td>3 ms</td><td>6 ms</td></tr></table> Compare Round Robin scheduling and Preemptive SJF based on average waiting time.	Process	Arrival Time	Burst Time	P1	0 ms	10 ms	P2	1 ms	4 ms	P3	2 ms	5 ms	P4	3 ms	6 ms	M					
Process	Arrival Time	Burst Time																				
P1	0 ms	10 ms																				
P2	1 ms	4 ms																				
P3	2 ms	5 ms																				
P4	3 ms	6 ms																				

21.	Analyse the impact of time quantum on the performance of Round Robin scheduling policy	H															
22.	A system experiences starvation under priority scheduling. Analyze why this occurs and how aging can resolve it.	H															
23.	A web server handles thousands of short-lived requests: • Which scheduling algorithm would you choose? • How would you tune it for optimal performance	H															
24.	If context switching had zero cost, how would scheduling strategies change? Analyze thoroughly	H															
25.	A system uses Round Robin with time quantum = 4, but switches to SJF-preemptive when queue length < 3. <table> <thead> <tr> <th>Process</th><th>Arrival Time</th><th>Burst Time</th></tr> </thead> <tbody> <tr> <td>P1</td><td>0</td><td>9</td></tr> <tr> <td>P2</td><td>1</td><td>5</td></tr> <tr> <td>P3</td><td>2</td><td>2</td></tr> <tr> <td>P4</td><td>4</td><td>7</td></tr> </tbody> </table> Analyze how hybrid scheduling affects performance.	Process	Arrival Time	Burst Time	P1	0	9	P2	1	5	P3	2	2	P4	4	7	H
Process	Arrival Time	Burst Time															
P1	0	9															
P2	1	5															
P3	2	2															
P4	4	7															
26.	A computer has two processors, $M1$ and $M2$. Four processes $P1, P2, P3, P4$ with CPU bursts of 20, 16, 25, and 10 milliseconds, respectively, arrive at the same time and these are the only processes in the system. The scheduler uses non-preemptive priority scheduling, with priorities decided as follows: • $M1$ uses priority of execution for the processes as, $P1 > P3 > P2 > P4$, i.e., $P1$ and $P4$ have highest and lowest priorities, respectively. • $M2$ uses priority of execution for the processes as, $P2 > P3 > P4 > P1$, i.e., $P2$ and $P1$ have highest and lowest priorities, respectively. A process P_i is scheduled to a processor M_k, if the processor is free and no other process P_j is waiting with higher priority. At any given point of time, a process can be allocated to any one of the free	H															

	processors without violating the execution priority rules. Ignore the context switch time. Find the average waiting time of the processes in milliseconds? [GATE-2025]	
27.	Processes P_1, P_2, P_3, P_4 arrive in that order at times 0, 1, 2, and 8 milliseconds respectively, and have execution times of 10, 13, 6, and 9 milliseconds respectively. Compare Shortest Remaining Time First (SRTF) - preemptive and nonpreemptive algorithms as CPU scheduling policy. Ignore context switching times. [GATE-2025]	M
28.	Consider a single processor system with four processes A, B, C, and D, represented as given below, where for each process the first value is its arrival time, and the second value is its CPU burst time. A (0, 10), B (2, 6), C (4, 3), and D (6, 7). Compare Shortest Remaining Time First (SRTF) - preemptive and nonpreemptive algorithms as CPU scheduling policy. Ignore context switching times. [GATE-2024]	M
29.	Consider a single processor system with four processes A, B, C, and D, represented as given below, where for each process the first value is its arrival time, and the second value is its CPU burst time. A (0, 10), B (2, 6), C (4, 3), and D (6, 7). Compare FCFS and nonpreemptive algorithms as CPU scheduling policy. Ignore context switching times.	M
30.	Consider a single processor system with four processes A, B, C, and D, represented as given below, where for each process the first value is its arrival time, and the second value is its CPU burst time. A (0, 10), B (2, 6), C (4, 3), and D (6, 7). Compare FCFS and preemptive algorithms as CPU scheduling policy. Ignore context switching times.	M
31	Consider a CPU that has to execute two types of processes. The first type, Actuators (A), requires a CPU burst of 6 seconds. The second type, Controllers (C), requires a CPU burst of 8 seconds. A new process of type A arrives at time $t = 10, 20, 30, 40$, and 50 (in seconds). Similarly, a new process of type C arrives at time $t = 11, 22, 33, 44$, and 55 (in seconds). The CPU scheduling policy is First Come First Serve (FCFS). The first process of type A starts running at $t = 10$ seconds. Calculate the average waiting time (in seconds) for	H

	the 10 processes. [GATE 2026]	
--	-------------------------------	--

Unit - 2 CO2 (Mrs. Mahalakshmi B.)		
Q. No.	Question	Level (L/M/H OTS)
1.	Identify the characteristic requirements for the solutions to the critical section problems.	L
2.	How memory barriers can be used to solve critical section problems?	L
3.	How can an 'atomic' variable be used to solve critical section problems?	L
4	Compare synchronous and asynchronous message passing mechanisms	L
5	How synchronization affects IPC?	L
6	Why does a deadlock occur in an operating system? Which are four necessary conditions for a deadlock to occur? Identify some real-world examples of deadlocks?	L
7	Why semaphores are necessary in IPC? Illustrate your answer with a simple example.	L
8	How to prevent deadlocks in semaphore-based IPC?	L
9	How can pipes be used for IPC?	M
10	How can shared memory be used for IPC?	M
11	How message passing can be used for IPC?	M
12	Compare Shared memory and Message passing mechanisms for IPC	M
13	How can readers-writers' problems be solved using shared memory?	M
14	How mutual exclusion can be implemented in hardware?	M
15	How does test_and_set help to implement mutual exclusion?	M
16	How compare_and_swap helps to implement mutual exclusion?	M

17	How does the producer-consumer problem lead to race conditions? Explain with code.	M
18	How mutex locks ensure process synchronization?	M
19	How interprocess communication impacts system security?	M
20	How does the shared memory mechanism overrides the operating system security mechanism?	M
21	Compare between busy-waiting and blocking synchronization in IPC.	M
22	Why privilege escalation is required during IPC?	M
23	When should message queues be used instead of shared memory for IPC? Explain in detail.	M
24	Compare semaphore and mutex	M
25	Compare pipes and shared memory mechanisms of IPC.	M
26	Does the following pseudo code result in a deadlock? Justify your answer. P1: lock(R1); lock(R2); P2: lock(R2); lock(R3); P3: lock(R3); lock(R1);	M
27	How does an operating system detect a deadlock? How often should deadlock detection be performed in a system?	M
28	Does the following pseudo code result in a deadlock? Justify your answer. P1: lock(A) ; lock(B); P2: lock(B) ; lock(A);	M
29	Race conditions are possible in many computer systems. Consider a banking system that maintains an account balance with two functions: deposit(amount) and withdraw(amount) . These two functions are passed the amount that is to be deposited or withdrawn from the bank account balance. Assume that a husband and wife share a bank account. Concurrently, the husband calls the withdraw() function, and the wife calls deposit() . Describe how a race condition is possible and what might be done to prevent the race condition from occurring.	M
30	Race conditions are possible in many computer systems. Consider an online auction system where the current highest bid for each item must be maintained. A person who wishes to bid on an item calls the bid(amount) function, which compares the amount being bid to the current highest bid. If the amount exceeds the current highest bid, the highest bid is set to the new amount. This is illustrated below:	M

	<pre>void bid(double amount) { if (amount > highestBid) highestBid = amount; }</pre> Describe how a race condition is possible in this situation and what might be done to prevent the race condition from occurring.													
31	How does the Banker's Algorithm help in avoiding deadlocks? What are the limitations of using the Banker's Algorithm?	M												
32	Consider the following code snippet and identify the issue causing a liveness failure (no progression), explaining your reasoning. <pre>wait(mutex); ... critical section ... wait(mutex);</pre>	M												
33	Analyse dining philosophers problem and the solution by using semaphore/mutex.	H												
34	Analyse producer-consumers problem and the solution by using semaphore/mutex.	H												
35	Analyse first reader-writers problem and the solution by using semaphore/mutex.	H												
36	Analyze the following synchronization solution, identify the possibility of deadlock, and design a deadlock-free version using any one prevention technique. Justify your approach. <pre>P1: lock(R1); lock(R2); P2: lock(R2); lock(R3); P3: lock(R3); lock(R1);</pre>	H												
37	Consider a system with 3 processes (P1, P2, P3) and 3 types of resources (R1, R2, R3). The available resources are (3, 3, 2). The allocated and maximum need matrices are: <table> <thead> <tr> <th>Process</th><th>Allocation (R1, R2, R3)</th><th>Max Need (R1, R2, R3)</th></tr> </thead> <tbody> <tr> <td>P1</td><td>(1, 0, 1)</td><td>(3, 2, 2)</td></tr> <tr> <td>P2</td><td>(2, 1, 1)</td><td>(4, 3, 3)</td></tr> <tr> <td>P3</td><td>(1, 1, 0)</td><td>(3, 2, 2)</td></tr> </tbody> </table>	Process	Allocation (R1, R2, R3)	Max Need (R1, R2, R3)	P1	(1, 0, 1)	(3, 2, 2)	P2	(2, 1, 1)	(4, 3, 3)	P3	(1, 1, 0)	(3, 2, 2)	H
Process	Allocation (R1, R2, R3)	Max Need (R1, R2, R3)												
P1	(1, 0, 1)	(3, 2, 2)												
P2	(2, 1, 1)	(4, 3, 3)												
P3	(1, 1, 0)	(3, 2, 2)												

	• Calculate the Need Matrix. • Determine if a request from P2 for (1,0,1) can be granted immediately without leading to an unsafe state.					
38	Consider the following two threads T1 and T2 that update two shared variables a and b. Assume that initially a=b=1. Though context switching between threads can happen at any time, each statement of T1 or T2 is executed atomically without interruption. <table border="1"><tr><td>T1 a=a+1; b=b+1;</td><td>T2 b=2*b; a=2*a;</td></tr></table> List all the possible combinations of values of a and b after both T1 and T2 finish execution?	T1 a=a+1; b=b+1;	T2 b=2*b; a=2*a;	H		
T1 a=a+1; b=b+1;	T2 b=2*b; a=2*a;					
39	A multithreaded web server wishes to keep track of the number of requests it services (known as hits). Consider the two following strategies to prevent a race condition on the variable hits. The first strategy is to use a basic mutex lock when updating hits, A second is to use an atomic integer as follows <table border="1"><tr><td>First strategy</td><td>Second strategy</td></tr><tr><td>int hits; mutex_lock hit_lock; hit_lock.acquire(); hits++; hit_lock.release();</td><td>atomic_t hits; atomic_inc(&hits);</td></tr></table> Critically evaluate the two approaches and justify which strategy provides better performance in a multithreaded environment.	First strategy	Second strategy	int hits; mutex_lock hit_lock; hit_lock.acquire(); hits++; hit_lock.release();	atomic_t hits; atomic_inc(&hits);	H
First strategy	Second strategy					
int hits; mutex_lock hit_lock; hit_lock.acquire(); hits++; hit_lock.release();	atomic_t hits; atomic_inc(&hits);					
40	Consider a multi-threaded program with two threads T1 and T2. The threads share two semaphores: s1 (initialized to 1) and s2 (initialized to 0). The threads also share a global variable x (initialized to 0). The threads execute the code shown below. <table border="1"><tr><td>Thread T1:</td><td>Thread T2:</td></tr><tr><td>wait(s1); x = x+1; print(x);</td><td>wait(s1); x = x+1; print(x);</td></tr></table>	Thread T1:	Thread T2:	wait(s1); x = x+1; print(x);	wait(s1); x = x+1; print(x);	H
Thread T1:	Thread T2:					
wait(s1); x = x+1; print(x);	wait(s1); x = x+1; print(x);					

	<table><tr><td>wait(s2); signal(s1);</td><td>signal(s2); signal(s1);</td></tr></table> Present which outcomes is/are possible when threads T1 and T2 execute concurrently?	wait(s2); signal(s1);	signal(s2); signal(s1);	
wait(s2); signal(s1);	signal(s2); signal(s1);			
41.	Consider three processes P1, P2, and P3 running identical code, as shown in the pseudocode below. A and B are two binary semaphores initialized to 1 and 0, respectively. X is a shared variable initialized to 0. Each line in the pseudocode is executed atomically. Pseudocode of P1, P2, and P3 <pre>Wait(A); Print(*); X = X+1; If (X == 2) { Print(\$); Signal(B); } Signal(A); Wait(B); Print(#); Signal(B);</pre> Assume that any of the three processes can start to execute first and context switching can happen between these processes at any arbitrary time and in any arbitrary order. Which patterns is/are possible to be generated as an outcome of the execution of these three processes? [GATE 2026]	H		

Unit - 3 CO3 (Mrs. Archana Kadam)		
Q. No.	Question	Level (L/M/H OTS)
1.	Summarize the differences between fixed and dynamic partitioning techniques.	L

2.	How logical addresses are translated into physical addresses in paging?	L																		
3.	Analyze the impact on system performance due to internal and external fragmentation	L																		
4.	How virtual memory is implemented in an operating system.	L																		
5.	Illustrate the concept of swapping and explain its role in memory management.	L																		
6.	Compare paging and segmentation w.r.t memory management	L																		
7.	Discuss the role of MMU (memory management unit) to map Logical Address to Physical Address	L																		
8.	Compare contiguous and non-contiguous memory allocation techniques.	L																		
	With a neat diagram, how swapping is done in memory management.	L																		
9	Illustrate how memory is allocated for partitions of sizes 300, 150, 100, 200, 20 with memory requests of sizes 80, 180, 280, 380, 30, and indicate the memory status after each request. Use first fit memory allocation technique	M																		
10	Apply demand paging concepts to show how a page table is used to translate logical addresses to physical addresses.	M																		
11	Consider the following reference string 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1. Assume there are three frames. Apply LRU replacement algorithm to the reference string above and find out how many page faults are produced	M																		
12	Consider the following segment Table <table border="1"> <thead> <tr> <th>Segment</th><th>Base</th><th>Length</th></tr> </thead> <tbody> <tr> <td>0</td><td>219</td><td>600</td></tr> <tr> <td>1</td><td>2300</td><td>14</td></tr> <tr> <td>2</td><td>90</td><td>100</td></tr> <tr> <td>3</td><td>1327</td><td>580</td></tr> <tr> <td>4</td><td>1952</td><td>96</td></tr> </tbody> </table> What are the physical addresses for the following logical addresses a. 0,430 b. 1, 10 c. 2, 500 d. 3, 400	Segment	Base	Length	0	219	600	1	2300	14	2	90	100	3	1327	580	4	1952	96	M
Segment	Base	Length																		
0	219	600																		
1	2300	14																		
2	90	100																		
3	1327	580																		
4	1952	96																		

13	Consider the following page reference string. 1,2,3,4,5,3,4,1,6,7,8,7,8,9,7,8,9,5,4,5,4,2 How many page faults would occur for the following replacement algorithm, assuming size of frame is four. a. page replacement. b. FIFO page replacement	M
14	Calculate page faults for (LRU, FIFO, OPT) for following sequences where page frame is three. 0,1,2,1,4,2,3,7,2,1,3,5,1,2,5.	M
15	Discuss the role of the Translation Lookaside Buffer (TLB) in virtual memory management.	M
16	A system has 8 GB of physical memory and a 32-bit virtual address space with a page size of 4 KB. (a) Calculate the number of entries in the page table. (b) Determine the size of the page table if each entry requires 4 bytes.	M
17	A process consists of four segments of sizes 2 KB, 6 KB, 4 KB, and 8 KB. The logical address space has a 4-bit segment number and a 12-bit offset. (a) Determine the maximum segment size. (b) Check whether an offset of 3000 bytes is valid for segment 1. Justify your answer.	M
18	A system has memory partitions of sizes 120 KB, 450 KB, 300 KB, 500 KB, and 700 KB. Four processes of sizes 350 KB, 275 KB, 150 KB, and 600 KB arrive for execution. (a) Apply the Worst Fit memory allocation strategy. (b) Calculate the total external fragmentation after allocation.	M
19	Compare and contrast the following Page Replacement Algorithms: FIFO (First In First Out), LRU (Least Recently Used) Optimal Page Replacement, Provide suitable examples to support your comparison.	M
20	Consider the following page reference string 7,2,3,1,2,5,3,4,6,7,7,1,0,5,4,6,2,3,0,1 Assume demand paging with three frames, how many page faults would occur for the following replacement algorithms? a. LRU replacement b. FIFO replacement	M
21	Consider the following page reference string: 0,4,1,4,2,3,4,1,3,2,40, 4, 1, 4, 2, 3, 4, 1, 3, 2, 40,4,1,4,2,3,4,1,3,2,4 Calculate the number of page faults using the Optimal, Page Replacement algorithm with 3 frames.	M
22	A process of 500 MB is swapped out to disk. With Disk speed = 100 MB/sec	M

	and Seek time = 20 ms Calculate the total time required for swapping out and swapping in the process.																																																																						
23	Consider a main memory with 4 page frames and the following sequence of page reference: 3,8,2,3,9,1,6,3,8,9,3,6,2,1,3 How many page faults would occur for the following replacement algorithms? Remember that all frames are initially empty. Rank these algorithms according to their page fault rate. LRU replacement FIFO replacement	M																																																																					
24	Consider a demand paging system with three frames, and the following page reference string: 1 2 3 4 5 4 1 6 4 5 1 3 2. The contents of the frames are as follows initially and after each reference (from left to right): <table><tr><td><i>initially</i></td><td colspan="12"><i>after</i></td></tr><tr><td>-</td><td>1*</td><td>2*</td><td>3*</td><td>4*</td><td>5*</td><td>4</td><td>1</td><td>6*</td><td>4</td><td>5</td><td>1*</td><td>3*</td><td>2*</td></tr><tr><td>-</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>1</td><td>6</td><td>6</td><td>6</td><td>6</td><td>6</td><td>2</td></tr><tr><td>-</td><td>-</td><td>2</td><td>2</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>4</td><td>1</td><td>1</td><td>1</td></tr><tr><td>-</td><td>-</td><td>-</td><td>3</td><td>3</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>5</td><td>3</td><td>3</td></tr></table> The *-marked references cause page replacements. Which page replacement policy/policies could be in use? [GATE-2025]	<i>initially</i>	<i>after</i>												-	1*	2*	3*	4*	5*	4	1	6*	4	5	1*	3*	2*	-	1	1	1	1	1	1	1	6	6	6	6	6	2	-	-	2	2	4	4	4	4	4	4	4	1	1	1	-	-	-	3	3	5	5	5	5	5	5	5	3	3	M
<i>initially</i>	<i>after</i>																																																																						
-	1*	2*	3*	4*	5*	4	1	6*	4	5	1*	3*	2*																																																										
-	1	1	1	1	1	1	1	6	6	6	6	6	2																																																										
-	-	2	2	4	4	4	4	4	4	4	1	1	1																																																										
-	-	-	3	3	5	5	5	5	5	5	5	3	3																																																										
25	A computer system has memory partitions of sizes 200 KB, 600 KB, 350 KB, 400 KB, and 800 KB. Four processes with memory requirements of 320 KB, 180 KB, 500 KB, and 250 KB arrive in the given order for execution. Apply following memory allocation techniques. First Fit Best Fit Worst Fit																																																																						
26	A system has memory partitions of sizes 120 KB, 450 KB, 300 KB, 500 KB, and 700 KB. Four processes of sizes 350 KB, 275 KB, 150 KB, and 600 KB arrive for execution. (a) Apply the First Fit memory allocation strategy. (b) Calculate the total external fragmentation after allocation																																																																						
27	A system has a TLB with an access time of 25 ns. The main memory access time is 150 ns. If the TLB hit ratio is 85%, calculate the Effective Access Time (EAT).	M																																																																					
28	Explain the concept of Demand Paging. Discuss its advantages and challenges. How does the system handle a Page Fault in a demand-paging environment?	M																																																																					

29	A system has a 32-bit virtual address space with a page size of 8 KB. The physical memory size is 512 MB. (a) Calculate the number of pages in the virtual address space. (b) Determine the number of bits required for the page number and the offset.	M
30	In a computer system, memory access time is 200 ns and TLB (Translation Lookaside Buffer) access time is 20 ns. The TLB hit ratio is 90%. Calculate the effective memory access time using the appropriate formula.	M
31	Explain the concept of Virtual Memory and its importance in modern operating systems. How does it differ from physical memory?	M
32	A system has memory partitions of sizes 250 KB, 500 KB, 350 KB, and 600 KB. Processes of sizes 200 KB, 425 KB, 300 KB, and 550 KB arrive for execution. (a) Apply the Best Fit memory allocation strategy to assign processes to partitions. (b) Calculate the total internal fragmentation after allocation.	M
33	How memory compaction can be used to manage external fragmentation. Provide a detailed analysis.	H
34	Consider a system with a 32-bit logical address and a 4 KB page size. a) Calculate the number of pages and the number of frames if the physical memory is 256 MB. b) Determine the number of bits required for the page number and the page offset.	H
35	A system uses virtual memory with a 40% page fault rate. The memory access time is 250 ns and the disk access time is 8 ms. Calculate the effective memory access time for the system using the appropriate formula. Provide all necessary steps and justify your result.	H
36	Given the following reference string: 1,2,3,4,2,1,5,6,2,1,2,3,7,1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7,1,2,3,4,2,1,5,6,2,1,2,3,7 Calculate the number of page faults using FIFO and LRU algorithms with 3 frames. Show all steps clearly.	H
37	A certain computer provides its user with a virtual memory space of 2^{32} bytes. The computer has 2^{35} bytes of physical memory. The virtual memory is implemented by paging the page size is 4096 bytes. A user process generates the virtual address 11123456. Explain how the system establishes the corresponding physical location.	H
38	Consider a logical address space of 256 pages with a 4KB page size, mapped onto physical memory of 64 frames. A. How many bits are required in the logical address? B. How many bits are required in the physical address	H

39	Free memory holes of sizes 15K, 10K, 5K, 25K, 30K, 40K are available. The processes of size 12K, 2K, 25K, 20K is to be allocated. How processes are placed in first fit, best fit, worst fit. Calculate internal as well as external fragmentation.	H
40	On a simple paging system with 2^{24} bytes of physical memory, 256 pages of logical address Space, and a, page size of 2^{10} bytes, how many bits are in logical address?	H
41.	Consider contiguous allocation of physical memory to processes using variable (dynamic) partitioning scheme. Suppose there are 8 holes in the memory of sizes 20 KB, 4 KB, 25 KB, 18 KB, 7 KB, 9 KB, 15 KB, and 12 KB. Assume that no two holes are adjacent. Two processes P1 of size 16 KB and P2 of size 9 KB arrive in that order. Compare performance of best-fit, first-fit and worst-fit based on the remaining number of holes of size less than 8 KB. [GATE- 2026 modified]	M
42.	In optimal page replacement algorithm, information about all future page references is available to the operating system (OS). A modification of the optimal page replacement algorithm is as follows: <i>The OS correctly predicts only up to next 4 page references (including the current page) at the time of allocating a frame to a page.</i> A process accesses the pages in the following order of page numbers: 1, 3, 2, 4, 2, 3, 1, 2, 4, 3, 1, 4. If the system has three memory frames that are initially empty, find the number of page faults that will occur during execution of the process. [GATE-2025]	M

Unit - 4 CO4 (Mrs. Dhanashree Patil & Dr. Pravin S. Game)		
Q. No.	Question	Level (L/M/H OTS)
1.	What are the different directory structures used in operating systems?How do they impact file retrieval?	L
2.	Compare different file allocation methods:contiguous,linked,indexed.What are their advantages and disadvantages?	M
3.	How does free-space management work in filesystems? Explain bitmap, linkedlist, and grouping methods.	L

4.	How does fragmentation affect free-space management?	M
a	Why disk scheduling is important in operating systems.	L
6.	How does the FIFO (First-Come-First-Served) disk scheduling algorithm work? What are its drawbacks?	M
7.	Compare SSTF (Shortest Seek Time First) with FIFO. How does SSTF reduce seek time?	M
8.	How does the SCAN (Elevator Algorithm) scheduling method work? Why is it preferred to SSTF?	M
9.	How does C-SCAN (Circular SCAN) differ from SCAN? What is its advantage?	M
10.	Identify the key factors to consider when selecting a disk scheduling algorithm for a given system?	L
11.	Consider two file systems A and B. that use contiguous allocation and linked allocation, respectively. A file of size 100 blocks is already stored in A and also in B. Now, consider inserting a new block in the middle of the file (between 50 and 51 block), whose data is already available in the memory. Assume that there are enough free blocks at the end of the file and that the file control blocks are already in memory. How many disk accesses are required to insert a block in the middle of the file in A and B respectively.	H
12.	Consider a disk drive with 4,000 cylinders, numbered from 0 to 3,999. The request queue has the following composition: 1045 750 932 878 1365 1787 1245 664 1678 1897. If the current position is 1167 and the previous request was served at 1250, compute the total distance (in cylinders) that the disk arm would move for each of the following algorithms: FIFO, SSTF, SCAN, and C-SCAN scheduling.	M
13.	Suppose that a disk drive has 5,000 cylinders, numbered 0 to 4,999. The drive is currently serving a request at cylinder 2,150, and the previous request was at cylinder 1,805. The queue of pending requests, in FIFO order, is: 2,069; 1,212; 2,296; 2,800; 544; 1,618; 356; 1,523; 4,965; 3,681 Starting from the current head position, what is the total distance (in cylinders) that the disk arm moves to satisfy all the pending requests for each of the following disk-scheduling algorithms? a. FCFS b. SCAN c. C-SCAN	M

	d. SSTF	
14.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF, SCAN and C-SCAN Request queue: 176, 79, 34, 60, 92, 11, 41, 114, 7, 150, 130, 20 Initial head: 50 Disk size: 0–199 Direction: towards 199	M
15.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF, SCAN and C-SCAN Request queue: 10, 180, 20, 170, 30, 160, 40, 150, 50, 140, 60, 130 Initial head: 75 Disk size: 0–199 Direction: towards 0	M
16.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF, SCAN and C-SCAN Request queue: 95, 96, 97, 98, 99, 100, 102, 104, 10, 15, 180, 185, 190 Initial head: 101 Disk size: 0–199 Direction: towards 199	M
17.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF, SCAN and C-SCAN Request queue: 5, 8, 12, 15, 18, 22, 25, 30, 35, 150, 160, 170, 180 Initial head: 90 Disk size: 0–199 Direction: towards 0	M
18.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF, SCAN and C-SCAN Request queue: 20, 40, 60, 80, 100, 120, 140, 160, 180, 10, 30, 50, 70, 90 Initial head: 85 Disk size: 0–199 Direction: towards 199	M

19.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF, SCAN and C-SCAN Request queue: 50, 52, 54, 56, 58, 60, 62, 64, 150, 170, 190, 10, 15 Initial head: 55 Disk size: 0–199 Direction: towards 199	M
20.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF, SCAN and C-SCAN Request queue: 38, 180, 130, 10, 50, 15, 190, 90, 150, 75, 65, 25, 5, 145 Initial head: 70 Disk size: 0–199 Direction: towards 199	M
21.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF, SCAN and C-SCAN Request queue: 12, 85, 190, 45, 60, 5, 155, 130, 75, 25, 95, 110, 180, 20, 140, 65 Initial head: 70 Disk size: 0–199 Direction: towards 199	M
22.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF, SCAN and C-SCAN Request queue: 3, 8, 15, 23, 42, 67, 89, 120, 145, 160, 175, 188 Initial head: 90 Disk size: 0–199 Direction: towards 0	M
23.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF, SCAN and C-SCAN Request queue: 40, 40, 41, 42, 100, 102, 103, 150, 150, 151, 10, 12, 14, 180 Initial head: 50 Disk size: 0–199 Direction: towards 199	M
24.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF.	M

	Request queue: 0, 1, 2, 3, 4, 195, 196, 197, 198, 199, 120, 130, 140 Initial head: 100 Disk size: 0–199 Direction: towards 199	
25.	Given Request queue, initial head, disk size and direction. Compare performance of FCFS, SSTF Request queue: 182, 45, 78, 10, 90, 150, 30, 60, 15, 200, 175, 5, 95, 140, 120, 35 Initial head: 85 Disk size: 0–200 Direction: towards 199	M
26.	Consider a system consisting of k instances of a resource R, being shared by 5 processes. Assume that each process requires a maximum of two instances of resource R and a process can request or release only one instance at a time. Further, a process can request the second instance of the resource only after acquiring the first instance. Find the minimum value of k for the system to be deadlock-free. (answer in integer) [GATE-2026]	H
27.	To keep track of free blocks in a file system, one of the two approaches is generally used – using bitmaps (bit vectors) or using linked lists. Consider that the linked list approach is used to keep track of free blocks in a file system. Assume that the disk size is 16 GB, block size is 2 KB, and block numbers used are 32-bit long. A single pointer of size 4 bytes is used in each block of the list to point to the next block of the list. Find the number of blocks required to hold the free disk block numbers. (answer in integer) Note: $1K=2^{10}$ and $1G=2^{30}$ [GATE-2026]	H
28.	A disk of size 512M bytes is divided into blocks of 64K bytes. A file is stored in the disk using linked allocation. In linked allocation, each data block reserves 4 bytes to store the pointer to the next data block. The link part of the last data block contains a NULL pointer (also of 4 bytes). Suppose a file of 1M bytes needs to be stored in the disk. Assume, $1K = 2^{10}$ and $1M = 2^{20}$. Calculate amount of space in bytes that will be wasted due to internal fragmentation. [GATE-2025]	M
29.	Given Request queue, initial head, disk size and direction. Compare performance of SCAN and C-SCAN Request queue: 0, 1, 2, 3, 4, 195, 196, 197, 198, 199, 120, 130, 140 Initial head: 100 Disk size: 0–199	M

	Direction: towards 199	
30.	Given Request queue, initial head, disk size and direction. Compare performance of SCAN and C-SCAN Request queue: 182, 45, 78, 10, 90, 150, 30, 60, 15, 200, 175, 5, 95, 140, 120, 35 Initial head: 85 Disk size: 0–200 Direction: towards 199	M